

```
Last login: Mon Aug  3 14:17:29 on ttys002
(eval):8: command not found: >
vishalkarnati@VISHALS-MacBook-Air ~ % gcc --version
Apple clang version 14.0.3 (clang-1403.0.22.14.1)
Target: arm64-apple-darwin25.2.0
Thread model: posix
InstalledDir: /Library/Developer/CommandLineTools/usr/bin
vishalkarnati@VISHALS-MacBook-Air ~ % git --version
git version 2.39.2 (Apple Git-143)
vishalkarnati@VISHALS-MacBook-Air ~ % cd ~/Desktop
vishalkarnati@VISHALS-MacBook-Air Desktop % touch file1.txt
vishalkarnati@VISHALS-MacBook-Air Desktop % cat Recursion/mazepath.java

public class mazepath {
```

```
    public static void path(int maze[][],
                            boolean visited[][],
                            int i,
                            int j,
                            int n,
                            String p) {

        // Out of bounds
        if(i < 0 || i >= n || j < 0 || j >= n)
            return;

        // Blocked cell
        if(maze[i][j] == 0)
            return;

        // Already visited
        if(visited[i][j])
            return;

        // Destination reached
        if(i == n-1 && j == n-1) {
            System.out.println(p + "(" + i + "," + j + ")");
            return;
        }

        visited[i][j] = true;

        // Right
        path(maze, visited,
            i, j+1, n,
            p + "(" + i + "," + j + ") -> ");

        // Down
        path(maze, visited,
            i+1, j, n,
            p + "(" + i + "," + j + ") -> ");

        // Left
        path(maze, visited,
            i, j-1, n,
            p + "(" + i + "," + j + ") -> ");

        // Up
        path(maze, visited,
```

```

// Right
path(maze, visited,
    i, j+1, n,
    p + "(" + i + ", " + j + ") -> ");

// Down
path(maze, visited,
    i+1, j, n,
    p + "(" + i + ", " + j + ") -> ");

// Left
path(maze, visited,
    i, j-1, n,
    p + "(" + i + ", " + j + ") -> ");

// Up
path(maze, visited,
    i-1, j, n,
    p + "(" + i + ", " + j + ") -> ");

// Backtracking
visited[i][j] = false;
}

public static void main(String args[]) {
    int maze[][] = {
        {1, 0, 0, 0},
        {1, 1, 0, 1},
        {0, 1, 0, 0},
        {1, 1, 1, 1}
    };

    boolean visited[][] = new boolean[4][4];

    path(maze, visited, 0, 0, 4, "");
}
}

```

```

vishalkarnati@VISHALS-MacBook-Air Desktop % mv file1.txt new.txt
vishalkarnati@VISHALS-MacBook-Air Desktop % rm new.txt
vishalkarnati@VISHALS-MacBook-Air Desktop % head Recursion/mazepath.java

```

```

public class mazepath {

    public static void path(int maze[][],
        boolean visited[][]),
vishalkarnati@VISHALS-MacBook-Air Desktop % tail Recursion/mazepath.java
    {0, 1, 0, 0},
    {1, 1, 1, 1}
    };

    boolean visited[][] = new boolean[4][4];

    path(maze, visited, 0, 0, 4, "");
}
}

```

```
vishalkarnati@VISHALS-MacBook-Air Desktop % more Recursion/mazepath.java
```

```
public class mazepath {

    public static void path(int maze[][],
                           boolean visited[][],
                           int i,
                           int j,
                           int n,
                           String p) {

        // Out of bounds
        if(i < 0 || i >= n || j < 0 || j >= n)
            return;

        // Blocked cell
        if(maze[i][j] == 0)
            return;

        // Already visited
        if(visited[i][j])
            return;

        // Destination reached
        if(i == n-1 && j == n-1) {
            System.out.println(p + "(" + i + "," + j + ")");
            return;
        }

        visited[i][j] = true;

        // Right
        path(maze, visited,
            i, j+1, n,
            p + "(" + i + "," + j + ") -> ");

        // Down
        path(maze, visited,
            i+1, j, n,
            p + "(" + i + "," + j + ") -> ");

        // Left
        path(maze, visited,
            i, j-1, n,
            p + "(" + i + "," + j + ") -> ");

        // Up
        path(maze, visited,
            i-1, j, n,
            p + "(" + i + "," + j + ") -> ");

        // Backtracking
        visited[i][j] = false;
    }

    public static void main(String args[]) {

        int maze[][] = {
```

```
            {1, 0, 0, 0},
            {1, 1, 0, 1},
            {0, 1, 0, 0},
            {1, 1, 1, 1}
        };

        boolean visited[][] = new boolean[4][4];

        path(maze, visited, 0, 0, 4, "");
    }
}
```

```
vishalkarnati@VISHALS-MacBook-Air Desktop % less Recursion/mazepath.java
```



```

public class mazepath {

    public static void path(int maze[],
                           boolean visited[],
                           int i,
                           int j,
                           int n,
                           String p) {

        // Out of bounds
        if(i < 0 || i >= n || j < 0 || j >= n)
            return;

        // Blocked cell
        if(maze[i][j] == 0)
            return;

        // Already visited
        if(visited[i][j])
            return;

        // Destination reached
        if(i == n-1 && j == n-1) {
            System.out.println(p + "(" + i + "," + j + ")");
            return;
        }

        visited[i][j] = true;

        // Right
        path(maze, visited,
            i, j+1, n,
            p + "(" + i + "," + j + ") -> ");

        // Down
        path(maze, visited,
            i+1, j, n,
            p + "(" + i + "," + j + ") -> ");

        // Left
        path(maze, visited,
            i, j-1, n,
            p + "(" + i + "," + j + ") -> ");

        // Up
        path(maze, visited,
            i-1, j, n,
            p + "(" + i + "," + j + ") -> ");

        // Backtracking
        visited[i][j] = false;
    }

    public static void main(String args[]) {

        int maze[][] = {
            {1, 0, 0, 0},

```

vishalkarnati@VISHALS-MacBook-Air Desktop % nano Recursion/mazepath.java

UW PICO 5.09

File: Recursion/mazepath.java

```

public class mazepath {

    public static void path(int maze[],
                           boolean visited[],
                           int i,
                           int j,
                           int n,
                           String p) {

        // Out of bounds
        if(i < 0 || i >= n || j < 0 || j >= n)
            return;

        // Blocked cell
        if(maze[i][j] == 0)
            return;

        // Already visited
        if(visited[i][j])
            return;

        // Destination reached
        if(i == n-1 && j == n-1) {
            System.out.println(p + "(" + i + "," + j + ")");
            return;
        }

        visited[i][j] = true;

        // Right
        path(maze, visited,
            i, j+1, n,
            p + "(" + i + "," + j + ") -> ");

        // Down
        path(maze, visited,
            i+1, j, n,
            p + "(" + i + "," + j + ") -> ");

        // Left
        path(maze, visited,
            i, j-1, n,
            p + "(" + i + "," + j + ") -> ");

        // Up
        path(maze, visited,
            i-1, j, n,
            p + "(" + i + "," + j + ") -> ");

        // Backtracking
        visited[i][j] = false;
    }
}

```